2026-7-27 Linux进程--消息队列与信号量

- 1. System V IPC 的共同基础
  - 1.1 为什么它们都属于 IPC
  - 1.2 key 是双方找到同一对象的约定
  - 1.3 内核如何统一管理 IPC 对象
  - 1.4 信号与信号量不是一回事
- 2. 消息队列
  - 2.1 消息队列是什么
  - 2.2 创建或获取消息队列
  - 2.3 发送与接收消息
    - 2.3.1 msgsnd：发送消息
    - 2.3.2 msgrcv：接收消息
  - 2.4 消息队列的控制与生命周期
  - 2.5 Server 与 Client 示例
- 3. System V 信号量
  - 3.1 为什么需要信号量
  - 3.2 信号量是资源计数器
    - 3.2.1 P 操作与 V 操作
    - 3.2.2 二元信号量与多元信号量
  - 3.3 为什么信号量本身也必须共享
  - 3.4 创建信号量集合
  - 3.5 初始化与控制信号量
  - 3.6 P/V 操作
  - 3.7 创建、初始化与使用示例
  - 3.8 生命周期与清理
- 4. 消息队列与信号量的关系
  - 4.1 数据传递与访问协调
  - 4.2 组合使用

1. System V IPC 的共同基础

1.1 为什么它们都属于 IPC

进程具有独立性，不能直接访问另一进程中的普通变量。要让进程协同工作，首先必须由操作系统提供一份双方都能看到的共享对象。

System V IPC 中常见的三类对象是：

- 共享内存：共享一段物理内存页。
- 消息队列：共享一个有类型的消息节点队列。
- 信号量：共享一组内核维护的计数器。

消息队列负责传递数据，信号量负责协调并发执行流的访问顺序。它们都可以通过 `key` 找到同一份内核资源，因此采用相似的创建、查询、控制和删除模型。

1.2 `key` 是双方找到同一对象的约定

使用 `ftok()` 根据路径和项目编号生成 key：

```
#include <sys/ipc.h>

key_t ftok(const char *pathname, int proj_id);
```

```
key_t key = ftok("/tmp", 0x66);
if (key == -1)
{
    perror("ftok");
    return 1;
}
```

通信双方使用同一个仍然存在的路径和同一个 `proj_id`，通常能得到相同的 key。key 用来在内核中找到同一类 IPC 对象，具体系统调用返回的 ID 则用于后续操作：

- `msqid`：消息队列 ID。
- `semid`：信号量集合 ID。
- `shmid`：共享内存 ID。

1.3 内核如何统一管理 IPC 对象

内核采用“先描述、再组织”的方式管理 IPC 资源。每个对象都具有两部分：

```
内核管理结构 + 资源本身
```

例如：

- 消息队列 = 队列管理结构 + 队列中的消息节点。
- 信号量集合 = 集合管理结构 + 一组信号量计数器。
- 共享内存 = 段管理结构 + 实际内存页。

不同对象共享一部分通用属性，例如 key、所有者、权限和序号，这些属性由 `ipc_perm` 这类通用结构描述；对象自己的大小、消息数量、挂接数或信号量个数则放在各自的属性结构中。

`struct shmid_ds` 是从共享内存对象中取出的一个具体属性结构示例：

```
struct shmid_ds {
    struct ipc_perm shm_perm;    /* Ownership and permissions */
    size_t          shm_segsz;   /* Size of segment (bytes) */
    time_t          shm_atime;   /* Last attach time */
    time_t          shm_dtime;   /* Last detach time */
    time_t          shm_ctime;   /* Creation time/time of last
                                    modification via shmctl() */
    pid_t           shm_cpid;    /* PID of creator */
    pid_t           shm_lpid;    /* PID of last shmat(2)/shmdt(2) */
    shmatt_t        shm_nattch;  /* No. of current attaches */
    ...
};
```

```
key = 0x66026002,_shmid = 10
共享内存的大小为: 128
共享内存的最后一次操作时间为: Mon Aug  3 09:21:48 2026
共享内存的最后一次附加时间为: Thu Jan  1 08:00:00 1970
共享内存的最后一次分离时间为: Mon Aug  3 09:21:40 2026
共享内存的当前附加进程数为: 2
```

可以用 `shmctl(shmid, IPC_STAT, ...)` 将内核保存的属性复制到用户空间：

```
shmctl(2)

NAME
       shmctl - System V shared memory control

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/shm.h>

       int shmctl(int shmid, int op, struct shmid_ds *buf);
```

这种“公共头部 + 具体对象”的设计说明：System V 的共享内存、消息队列和信号量虽然使用不同系统调用，但创建方式、权限、key、生命周期和删除方式具有共同规律。

1.4 信号与信号量不是一回事

课堂板书特别强调：

> `Linux 信号`与`Linux 信号量`没有直接关系。

- 信号是外部事件或硬件向进程发送的异步通知，例如 `SIGINT`、`SIGPIPE`。
- 信号量是用于同步与互斥的资源计数器。

本笔记只讨论 System V 信号量，不展开 Linux 进程信号的处理机制。

2. 消息队列

2.1 消息队列是什么

管道是面向字节流的，不保留消息边界；消息队列把数据组织成一个一个带类型的消息节点：

```
进程 A                          进程 B
  |                               |
  +--> [消息 1][消息 2][消息 3] --+
         type A   type B   type A
```

每个消息节点至少包含：

1. 一个正数类型的 `mtype`。
2. 一段消息正文。

接收方既可以按队列顺序接收，也可以按消息类型选择接收。因此，消息队列比纯字节流多了“消息边界”和“类型筛选”两项能力。

消息队列是一个共享的队列头，队列头管理若干消息节点。不同进程通过相同 key 和相同 `msqid` 找到同一个队列。

2.2 创建或获取消息队列

```
msgget(2)

NAME
       msgget - get a System V message queue identifier

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/msg.h>

       int msgget(key_t key, int msgflg);
```

```
#include <sys/ipc.h>
#include <sys/msg.h>

int msgget(key_t key, int msgflg);
```

- 成功返回非负的 `msqid`。
- 失败返回 `-1`，并设置 `errno`。
- `IPC_CREAT`：不存在则创建，存在则获取。
- `IPC_CREAT | IPC_EXCL`：不存在则创建，已存在则失败。
- 权限位仍可使用 `0666` 等模式。

```
int msqid = msgget(key, IPC_CREAT | 0666);
```

需要确保自己创建的是全新队列时：

```
int msqid = msgget(key, IPC_CREAT | IPC_EXCL | 0666);
```

若返回 `EEXIST`，说明同一 key 对应的消息队列已经存在，不是 `msqid` 冲突。

查看系统中的消息队列：

```
ipcs -q
```

```
ZL@VM-0-14-ubuntu:~$ ipcs -q

------ Message Queues --------
key        msqid      owner      perms      used-bytes   messages
```

2.3 发送与接收消息

用户先定义自己的消息类型：

```
struct msgbuf
{
    long mtype;       // 必须为正数
    char mtext[1024]; // 消息正文
};
```

2.3.1 `msgsnd`：发送消息

```
int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
```

- `msgp`：指向用户定义的消息结构。
- `msgsz`：正文长度，不包含 `mtype` 的大小。
- 成功返回 `0`，失败返回 `-1`。
- 队列达到容量或资源限制时，默认阻塞等待。
- `IPC_NOWAIT` 可让无法发送时立即返回。

```
Message msg{};
msg.mtype = 1;
std::strncpy(msg.mtext, "hello", sizeof(msg.mtext) - 1);
msgsnd(msqid, &msg, std::strlen(msg.mtext), 0);
```

2.3.2 `msgrcv`：接收消息

```
ssize_t msgrcv(int msqid, void *msgp, size_t msgsz,
               long msgtyp, int msgflg);
```

`msgtyp` 决定选择哪条消息：

- `msgtyp == 0`：接收队列中的第一条消息。
- `msgtyp > 0`：接收第一条指定类型的消息。
- `msgtyp < 0`：接收类型值不大于 `|msgtyp|` 的第一条消息。

常用标志：

- `IPC_NOWAIT`：没有符合条件的消息时不阻塞。
- `MSG_NOERROR`：正文过长时截断，而不是返回错误。

2.4 消息队列的控制与生命周期

```
msgctl(2)

NAME
       msgctl - System V message control operations

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/msg.h>

       int msgctl(int msqid, int op, struct msqid_ds *buf);
```

```
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

常用命令：

- `IPC_STAT`：读取队列属性。
- `IPC_SET`：修改允许变更的属性。
- `IPC_RMID`：删除消息队列。

```
msgctl(msqid, IPC_RMID, nullptr);
```

也可以使用命令行：

```
ipcrm -q <msqid>
```

System V IPC 资源的生命周期跟随内核。创建进程退出不会自动删除消息队列；若双方不再使用，应显式调用 `IPC_RMID` 或执行 `ipcrm`，否则资源可能一直保留到系统重启。

2.5 Server 与 Client 示例

下面约定类型 `1` 表示客户端发往服务端的消息：

```
#include <cerrno>
#include <cstdio>
#include <cstring>
#include <iostream>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <unistd.h>

constexpr long CLIENT_TO_SERVER = 1;

struct Message
{
    long mtype;
    char mtext[256];
};
```

服务端：

```
int main()
{
    key_t key = ftok("/tmp", 0x66);
    int msqid = msgget(key, IPC_CREAT | IPC_EXCL | 0666);
    if (msqid == -1)
    {
        std::perror("msgget");
        return 1;
    }

    Message msg{};
    if (msgrcv(msqid, &msg, sizeof(msg.mtext), CLIENT_TO_SERVER, 0) > 0)
    {
        std::cout << "server received: " << msg.mtext << '\n';
    }

    msgctl(msqid, IPC_RMID, nullptr);
    return 0;
}
```

客户端：

```cpp
int main()
{
    key_t key = ftok("/tmp", 0x66);
    int msqid = msgget(key, 0);
    if (msqid == -1)
    {
        std::perror("msgget");
        return 1;
    }

    Message msg{};
    msg.mtype = CLIENT_TO_SERVER;
    std::strncpy(msg.mtext, "hello from client", sizeof(msg.mtext) - 1);
    msgsnd(msqid, &msg, std::strlen(msg.mtext), 0);
    return 0;
}
```

这两个程序应分别编译，并在服务端建立队列后运行客户端。实际工程还应检查 `msgsnd`、`msgrcv` 的返回值，并约定消息正文的终止符或长度字段。

3. System V 信号量

3.1 为什么需要信号量

多个进程通过共享内存或消息队列协同工作时，会出现并发访问问题：

- 一个进程写了一半，另一个进程就开始读。
- 两个进程同时修改同一份状态。
- 输出流中的内容互相穿插。

即使 `fork()` 后的父子进程同时 `printf()`，输出也可能混杂。原因不是显示器本身出了问题，而是两个执行流同时访问同一个共享输出资源，却没有顺序保护。

需要区分几个概念：

- **共享资源**：多个执行流能够同时看到并访问的公共资源。
- **临界资源**：同一时刻只允许一个执行流访问的共享资源。
- **临界区**：进程中访问临界资源的那段代码。
- **互斥**：任意时刻只允许一个执行流进入临界区。
- **同步**：多个执行流按照约定顺序协调执行。

- **原子性**：操作要么不执行，要么完整执行，不会停在中间状态。

共享资源应被保护，保护的本质是约束“访问共享资源的代码段”。信号量就是把这种约束落实到 P/V 操作上的内核机制。

3.2 信号量是资源计数器

信号量不是 Linux 信号，也不是普通的用户态整数。它由内核维护，本质是一个计数器，用数值描述当前可用的资源数量。

3.2.1 P 操作与 V 操作

```
P 操作：申请一个资源，计数器减一
V 操作：释放一个资源，计数器加一
```

申请流程可以概括为：

1. 先申请信号量。
2. 计数允许时，成功预定资源。
3. 计数不允许时，进程阻塞等待。
4. 使用资源。
5. 释放资源并唤醒等待者。

信号量是一个“资源预定机制”。买电影票时，票的数量对应信号量；买到票后座位被提前预定，但真正使用座位仍由后续流程完成。

3.2.2 二元信号量与多元信号量

- **二元信号量**：计数范围只有 `0` 和 `1`，一次只允许一个执行流进入临界区，因此可用于互斥。
- **多元信号量**：计数表示多个同类资源，例如 10 个座位、5 个缓冲区槽位。

无论哪种形式，P/V 都必须是原子操作。否则两个进程同时执行“判断并减一”时，会产生竞争，计数器本身反而成为新的共享资源冲突点。

3.3 为什么信号量本身也必须共享

若两个进程要协调同一份资源，它们必须先看到同一个信号量。不同信号量不会产生协调效果。

因此通信双方需要：

1. 使用相同 key 找到同一个信号量集合。
2. 对指定集合中的同一个信号量执行 P/V。
3. 由内核原子地完成计数检查和修改。

信号量本身是共享资源，但内核保证 P/V 的检查与更新不可被其他 P/V 插入，因此不会像普通整数那样出现两个人同时“看到 1 都减成 0”的竞争。

3.4 创建信号量集合

```
semget(2)

NAME
       semget - get a System V semaphore set identifier

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/sem.h>

       int semget(key_t key, int nsems, int semflg);
```

```c
#include <sys/ipc.h>
#include <sys/sem.h>

int semget(key_t key, int nsems, int semflg);
```

- 成功返回非负的 `semid`。
- 失败返回 `-1`，并设置 `errno`。
- `nsems`：集合中包含多少个信号量，不是所有信号量初始值之和。
- `IPC_CREAT`：不存在则创建，存在则获取。
- `IPC_CREAT | IPC_EXCL`：要求创建一个新集合。

```c
int semid = semget(key, 1, IPC_CREAT | IPC_EXCL | 0666);
```

例如 `nsems = 4` 表示创建一个有 4 个独立信号量的集合，每个信号量都需要单独设置初值和执行操作。

查看信号量集合：

```
ipcs -s
```

删除信号量集合：

```
ipcrm -s <semid>
```

3.5 初始化与控制信号量

```
msgctl(2)

NAME
       msgctl - System V message control operations

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <sys/msg.h>

       int msgctl(int msqid, int op, struct msqid_ds *buf);
```

```
int semctl(int semid, int semnum, int cmd, ...);
```

该函数是可变参数接口，常用控制命令包括：

- `SETVAL`：设置集合中某个信号量的值。
- `GETVAL`：读取某个信号量的值。
- `IPC_STAT`：读取集合属性。
- `IPC_RMID`：删除整个信号量集合。

为 `SETVAL` 传值通常需要自定义联合体：

```
union Semun
{
    int val;
```

```
    struct semid_ds *buf;
    unsigned short *array;
};

Semun arg{};
arg.val = 1;

if (semctl(semid, 0, SETVAL, arg) == -1)
    perror("semctl SETVAL");
```

初始化与创建分开的原因之一是：`semget()` 只负责创建或获取集合，不直接规定每个信号量的初始计数。执行 `IPC_RMID` 时：

```
semctl(semid, 0, IPC_RMID);
```

`IPC_RMID` 删除的是整个信号量集合，不只是编号为 `0` 的那个信号量。

3.6 P/V 操作

System V 使用 `semop()` 执行一个或多个原子操作：

```
int semop(int semid, struct sembuf *sops, size_t nsops);
```

```
struct sembuf
{
    unsigned short sem_num; // 集合中的信号量编号
    short sem_op;           // 操作量
    short sem_flg;          // 操作标志
};
```

`sem_op` 的含义：

- `sem_op < 0`：申请 `|sem_op|` 个资源，相当于 P 操作；资源不足时阻塞。
- `sem_op == 0`：等待该信号量的值变为 `0`。
- `sem_op > 0`：释放 `sem_op` 个资源，相当于 V 操作，并唤醒等待者。

例如二元信号量：

```
sembuf pOperation{0, -1, SEM_UNDO};
semop(semid, &pOperation, 1); // P

// 访问临界区

sembuf vOperation{0, 1, SEM_UNDO};
semop(semid, &vOperation, 1); // V
```

`SEM_UNDO` 让系统在进程异常退出时尝试撤销该进程尚未恢复的信号量操作，适合降低异常退出留下的计数问题风险。`IPC_NOWAIT` 则表示无法立即完成时不阻塞，而是返回错误。

3.7 创建、初始化与使用示例

下面的例子用二元信号量保护一个临界区：

```
#include <cerrno>
#include <cstdio>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <unistd.h>

union Semun
{
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};

int main()
{
    key_t key = ftok("/tmp", 0x66);

    int semid = semget(key, 1, IPC_CREAT | IPC_EXCL | 0666);
    if (semid != -1)
    {
        Semun arg{};
        arg.val = 1; // 二元信号量初始为 1
        if (semctl(semid, 0, SETVAL, arg) == -1)
        {
            std::perror("semctl");
            return 1;
        }
    }
```

```
    else if (errno == EEXIST)
    {
        semid = semget(key, 1, 0);
    }

    if (semid == -1)
    {
        std::perror("semget");
        return 1;
    }

    sembuf pOperation{0, -1, SEM_UNDO};
    sembuf vOperation{0, 1, SEM_UNDO};

    if (semop(semid, &pOperation, 1) == -1)
    {
        std::perror("P");
        return 1;
    }

    std::puts("enter critical section");
    sleep(2);
    std::puts("leave critical section");

    if (semop(semid, &vOperation, 1) == -1)
        std::perror("V");

    return 0;
}
```

这段代码演示单个进程的 P/V 方向。真正让两个进程互斥时，应让双方使用相同 key 和相同 `semid`，并在访问共享资源之前执行 P、之后执行 V。

创建并初始化信号量时应避免多个进程同时执行 `SETVAL`，否则可能把已经被使用的计数重置。常见做法是：

1. 由明确的创建者使用 `IPC_CREAT | IPC_EXCL` 创建集合。
2. 创建者在其他参与者开始使用前完成 `SETVAL`。
3. 其他参与者只使用 `semget(key, nsems, 0)` 获取已有集合。

3.8 生命周期与清理

信号量和消息队列一样，生命周期跟随内核：

- 创建进程退出不会自动删除信号量集合。
- 未显式删除的资源可能一直存在到系统重启。
- 使用 `ipcrm -s <semid>` 或 `semctl(semid, 0, IPC_RMID)` 删除。
- 删除前应确保等待者和其他使用者已经退出，避免其操作到不存在的集合。

4. 消息队列与信号量的关系

4.1 数据传递与访问协调

对比项	System V 消息队列	System V 信号量
资源本身	有类型的消息节点队列	一组内核计数器
核心用途	传递数据和控制消息	同步、互斥与资源预定
基本操作	`msgsnd`、`msgrcv`	`semop` 的 P/V
是否保留消息边界	是	不传递消息正文
选择能力	可按 `mtype` 选择消息	可按集合编号选择信号量
创建接口	`msgget`	`semget`
控制接口	`msgctl`	`semctl`
主要 ID	`msqid`	`semid`
生命周期	随内核，需显式删除	随内核，需显式删除

4.2 组合使用

消息队列可以发送任务或通知，信号量可以限制资源数量和访问顺序。例如：

- 用二元信号量保护共享内存中的配置结构。
- 用多元信号量限制缓冲区中可写槽位的数量。
- 用消息队列发送“任务类型 + 任务参数”，由工作进程按类型接收。

System V 的 `shmget`、`msgget`、`semget` 以及对应的 `*ctl` 接口之所以相似，是因为它们共同遵循一套内核资源管理模型。理解这套模型后，重点就不再只是记住函数名，而是明确每个 IPC 对象承载什么资源、由谁创建、如何找到、何时释放，以及并发访问由谁负责协调。